PROJECT REPORT

VoltRelay Data Analysis

Battery Swapping Network Analytics

A data analytics project that examines battery-swapping network performance, service failures, station patterns, battery health, pricing, and rider retention to produce actionable business insights.

Project Title	VoltRelay Energy — Data Analytics (Battery Swapping Network)
Repository	github.com/tanmayikt98-coder/VoltRelay-Data-Analysis

1. Project Purpose

VoltRelay Data Analysis is a Jupyter Notebook–based analytics project for a battery-swapping network (VoltRelay Energy). It studies how the network performs over time, where service failures occur, how stations and batteries behave, how pricing and partners contribute to economics, and how riders return or churn. The goal is to turn operational data into clear, actionable business insights.

Battery-swapping operators collect large volumes of event, station, battery and support data, but without structured analysis it is hard to see trends in failures, revenue or retention. This project addresses that need by cleaning multi-table datasets, computing key performance indicators, and visualising patterns that management can act on.

The project was chosen because it applies core data-analysis skills (cleaning, aggregation, visualisation and KPI design) to a realistic operational domain. Target users are students learning data analytics and operators or managers who need a transparent view of network health. The main objective is to deliver analysis across six areas: network performance over time, service failures and customer experience, station and geographic patterns, battery and equipment performance, pricing and partner economics, and rider retention and churn.

2. Technologies Used

The following technologies are used in the actual notebook:

• Python — Main programming language for all analysis steps.
• Pandas — Loading CSV datasets, cleaning invalid values, grouping, merging and aggregating.
• NumPy — Numerical operations and handling of missing or invalid measurements.
• Matplotlib — Line charts, bar charts and other plots for trends and comparisons.
• Seaborn — Additional statistical and categorical visualisations.
• Jupyter Notebook / Google Colab — Interactive environment where the full analysis is written and run.
• CSV data files — Source tables such as swap_events, stations, batteries, riders, support_tickets, station_hourly_status, city_daily_context and fleet_partners.

These tools were chosen because they are standard for data analytics coursework and industry practice. Pandas and Matplotlib allow transparent, step-by-step work that is easy to explain in a viva, while Colab makes the notebook runnable without a local setup.

3. Working and Logic

Workflow

1. Multiple CSV files are loaded (swap events, stations, batteries, riders, support tickets, etc.).
2. Swap events are cleaned: negative kilometres, SOC/SOH values above 100%, and similar anomalies are set to missing; event categories (Completed, Failed, Abandoned, Cancelled) are derived.
3. Timestamps are converted to datetime and a month column is created for time-series views.
4. Aggregations produce monthly performance (attempts, completions, failures, revenue), failure-reason breakdowns, and station-level failure and abandonment rates.
5. Station, battery and support-ticket data are joined where needed to study geography, equipment health and customer complaints.
6. Rider activity is used to classify returners vs one-time users and to summarise retention.
7. Final KPI table summarises total attempts, completed swaps, failure and abandonment rates, revenue, contribution margin, ticket volume and average CSAT.

User Input Processing

Input is provided as pre-existing CSV datasets placed in the Colab (or local) working directory. There is no interactive form; the analyst runs cells in sequence. Paths such as /content/*.csv are used in the notebook as published, which matches a typical Colab upload workflow.

Analysis / Decision Logic

Event categories are assigned from raw event types so that success, failure, abandonment and cancellation can be counted consistently. Rates (failure rate, abandonment rate) are calculated as counts divided by total attempts. Station and connectivity-tier comparisons highlight where problems concentrate. Rider retention is based on whether a rider has more than one completed

swap. Support tickets are aggregated by station for complaint volume, resolution time and CSAT. No machine-learning classifier is required; the project relies on cleaning, aggregation and clear KPI definitions.

Main Components

• Data loading cells — Read all operational CSVs into Pandas DataFrames.
• Cleaning pipeline — Correct invalid km/SOC/SOH values and build event categories.
• Time-series analysis — Monthly completed swaps, failure rates and revenue trends.
• Failure analysis — Reason codes and station-level failure rankings.
• Station / connectivity analysis — Performance by location type and connectivity tier.
• Rider retention — First/last swap and return-status classification.
• Support and KPI summary — Ticket hotspots and a consolidated KPI table.

Output

The notebook produces printed tables, Matplotlib/Seaborn charts (monthly trends, failure reasons, station comparisons) and a final KPI summary covering attempts, completions, failure and abandonment rates, revenue, average contribution margin per swap, support ticket count and average CSAT. These outputs support business discussion of reliability, economics and retention.

4. Testing and Limitations

No separate automated test suite is present. Correctness is checked by inspecting shapes, missing-value counts, date ranges and sample aggregations inside the notebook. Charts and KPI tables are reviewed manually after each major analysis block. The notebook is intended to be run cell-by-cell in Colab or a local Jupyter environment with the CSV files available.

Challenges Observed

• Invalid numeric values (negative km, SOC/SOH above 100%) required explicit cleaning rules.
• Multiple related tables (events, stations, tickets, riders) must be joined carefully to avoid double-counting or mismatched keys.
• Paths in the notebook point to /content/, so local runs need path adjustments or the same folder layout.

Limitations

• The repository contains only the notebook and a minimal README; the CSV source files are not published in the repo and must be supplied separately (e.g. uploaded to Colab).
• Analysis is exploratory and descriptive; it does not include predictive models or automated alerts.
• Results depend on the quality and completeness of the underlying operational extracts.
• Timestamp correction for a documented firmware issue was not applied because the dataset date

range does not overlap the affected period.

Error Handling

Invalid measurements are converted to NaN rather than left as extreme values. Timestamp conversion uses errors='coerce' so bad values become missing. Group-by and merge steps assume consistent keys; missing joins simply reduce the matched set. The notebook does not implement try/except around every cell; the analyst is expected to resolve path or schema issues when they appear.

Possible Future Improvements

• Publish sample or anonymised CSV files alongside the notebook for fully reproducible runs.
• Add a short requirements.txt and path-configuration cell for local execution.
• Extend analysis with simple forecasting or anomaly detection on failure rates.
• Produce a one-page executive dashboard summary in addition to detailed cells.

5. GitHub and Originality

Repository Structure

```
VoltRelay-Data-Analysis/
├── README.md                    (title only)
└── VoltRelay_Data_Analysis.ipynb  (full analysis notebook, ~155 cells)
```

The notebook is configured with a Colab badge and loads CSVs from /content/. Source data files are not included in the GitHub repository itself.

Execution Steps

1. Open the notebook in Google Colab (via the badge) or in a local Jupyter environment with Python, Pandas, NumPy, Matplotlib and Seaborn installed.
2. Upload or place the required CSV files (swap_events, stations, batteries, riders, support_tickets, station_hourly_status, city_daily_context, fleet_partners) where the notebook expects them.
3. Run cells from top to bottom. Cleaning, aggregations and plots will execute in sequence.
4. Review the final KPI table and charts for business conclusions.

Individual Contributions

I contributed to the overall project development, including data cleaning rules, performance and failure analysis, station and retention metrics, visualisation design, KPI summary, and preparation of the work for academic submission.

AI Assistance

AI tools were used for coding assistance, structuring analysis steps, and preparing documentation during development and report writing.

Learning Outcomes

- Cleaning multi-column operational data and defining consistent event categories.
- Building time-based and station-based aggregations for reliability and revenue.
- Linking support tickets and rider activity to operational events.
- Designing a small set of clear KPIs that summarise network health.
- Presenting results with plots and tables suitable for both technical and business audiences.

Conclusion

VoltRelay Data Analysis shows how a battery-swapping network’s operational data can be turned into structured insights. Through careful cleaning, monthly and station-level aggregations, failure-reason breakdowns, retention views and a final KPI summary, the notebook provides a practical template for reliability, economics and customer-experience analysis. With the source CSVs available, it is a complete and explainable analytics project suitable for academic demonstration and for guiding operational discussion.

— End of Report —